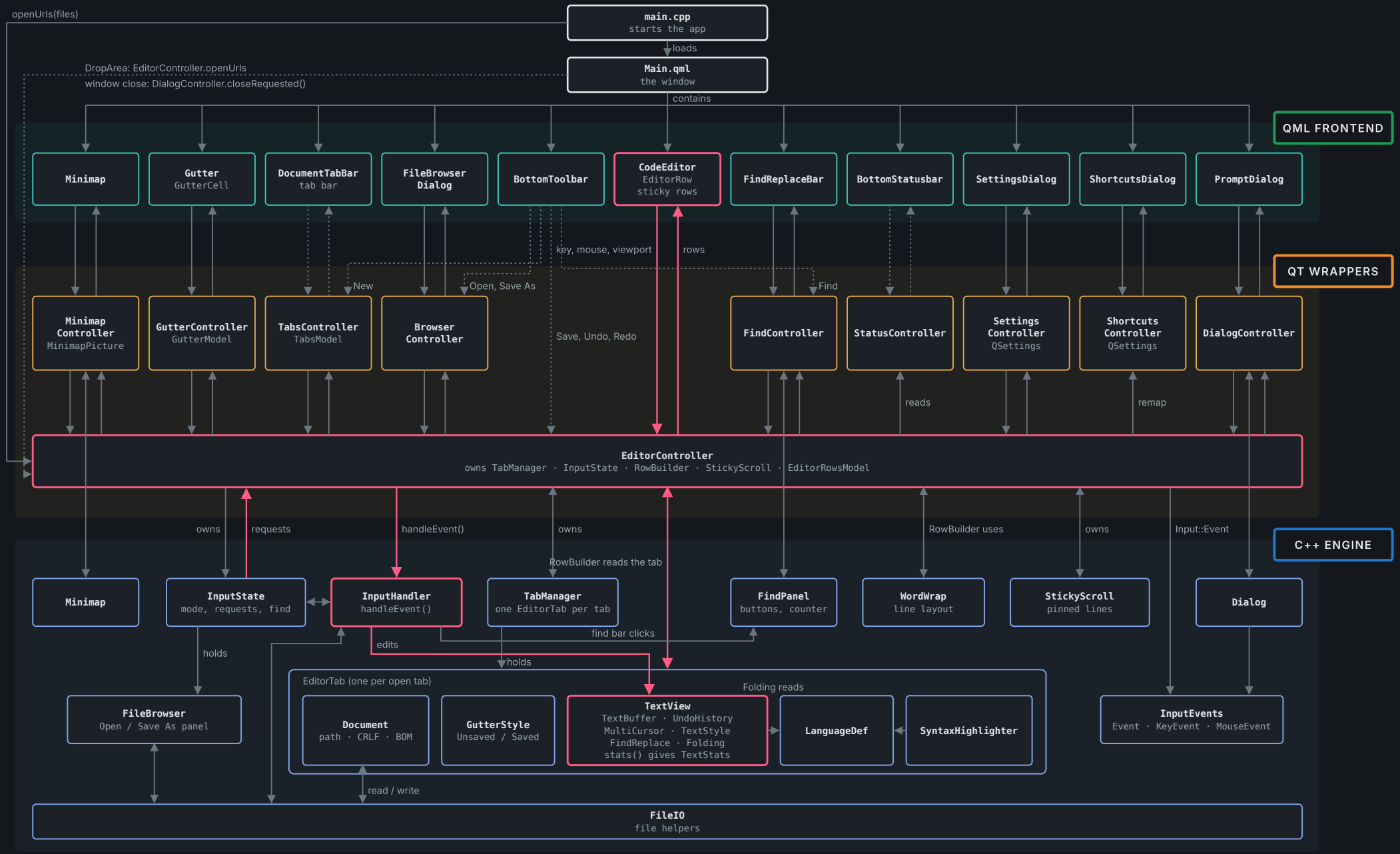
ChaoticNotepad Map

one arrow: from the box that calls, owns or reads to the box it uses (results come back as return values)

two arrows: calls and events one way, models and signals back

main path of typing

wiring done in Main.qml



Reading the map

The **red path** is every keystroke. Every box and link was checked against your full source: headers, `.cpp` files, `#include` lines and QML files.

The hub

- **`EditorController`** owns `TabManager`, `InputState`, `RowBuilder`, `StickyScroll` and `EditorRowsModel`.
- Every controller except `ShortcutsController` calls it.
- It calls **`ShortcutsController`** to remap each key before the engine sees it, and **`DialogController`** for quit, close and save conflicts.
- `MinimapController` owns the engine **`Minimap`**, `FindController` uses **`FindPanel`**, and `DialogController` owns the engine **`Dialog`**. Their straight lines run over the hub box, not through it.
- `TabsController` asks `DialogController` before closing a tab.

Dotted lines: wiring done in `Main.qml`

- **BottomToolbar clicks:** New to `TabsController`; Open and Save As to `BrowserController`; Find to `FindController`; Save, Undo and Redo to `EditorController`.
- Tab bar and `TabsController`.
- Status bar and `StatusController`.
- Dropped files to **`EditorController.openUrls`**.

Also in `Main.qml`, not drawn as lines

- Window close calls **`DialogController.closeRequested`** (the note beside `Main.qml`).
- The Settings and Shortcuts buttons open their dialogs.
- `Gutter` and `Minimap` pass scrolling to `CodeEditor`.
- A closing bar or dialog gives the focus back to `CodeEditor`.

Wrappers

- `BrowserController` reaches the **`FileBrowser`** held by `InputState`.
- `SettingsController` applies indentation and bracket closing to every tab through `TabManager`, and the minimap switch in `InputState`.
- `StatusController` only reads.

Reading the map, continued, and one keystroke

Inside the engine

- **`handleEvent`** works on a `TextView` and the `InputState`, using the events in `InputEvents`.
- **`RowBuilder`** reads the current `EditorTab` and uses `WordWrap` and `StickyScroll`.
- Each **`EditorTab`** holds one `TextView`, `Document`, `GutterStyle` and `SyntaxHighlighter`.
- **`GutterStyle`** only works out which lines are unsaved, saved or untouched. `TabManager` resets it when a file opens, `EditorController` updates it and marks it saved, and `RowBuilder` and `MinimapController` read the marks for the gutter and the minimap.
- **`TextView`** owns the buffer, undo history, cursors, formatting, find and folding, and `stats()` gives a `TextStats`.
- `Folding` and `SyntaxHighlighter` both read **`LanguageDef`**.

Files and find

- `Document` loads and saves through `TextView`.
- **`FileIO`** is used by `Document`, `FileBrowser` and `InputHandler`. Those links are two-way: the caller sends a path or text, and `FileIO` hands back the file contents, a listing, a result or an error.
- `InputHandler` also uses `FindPanel` for clicks on the find bar buttons.

Arrow rules

- Every arrow from `EditorController` into the engine is **two-way** (it calls, and reads what comes back), except two: **`handleEvent`** returns nothing, because the result is left in the `TextView` and `InputState`, and **`InputEvents`** only goes in.

Left out

- The engine **`Gutter`** namespace (width and label helpers) and **`TabBar`** (the tab strip of a terminal front end). Nothing includes either of them.

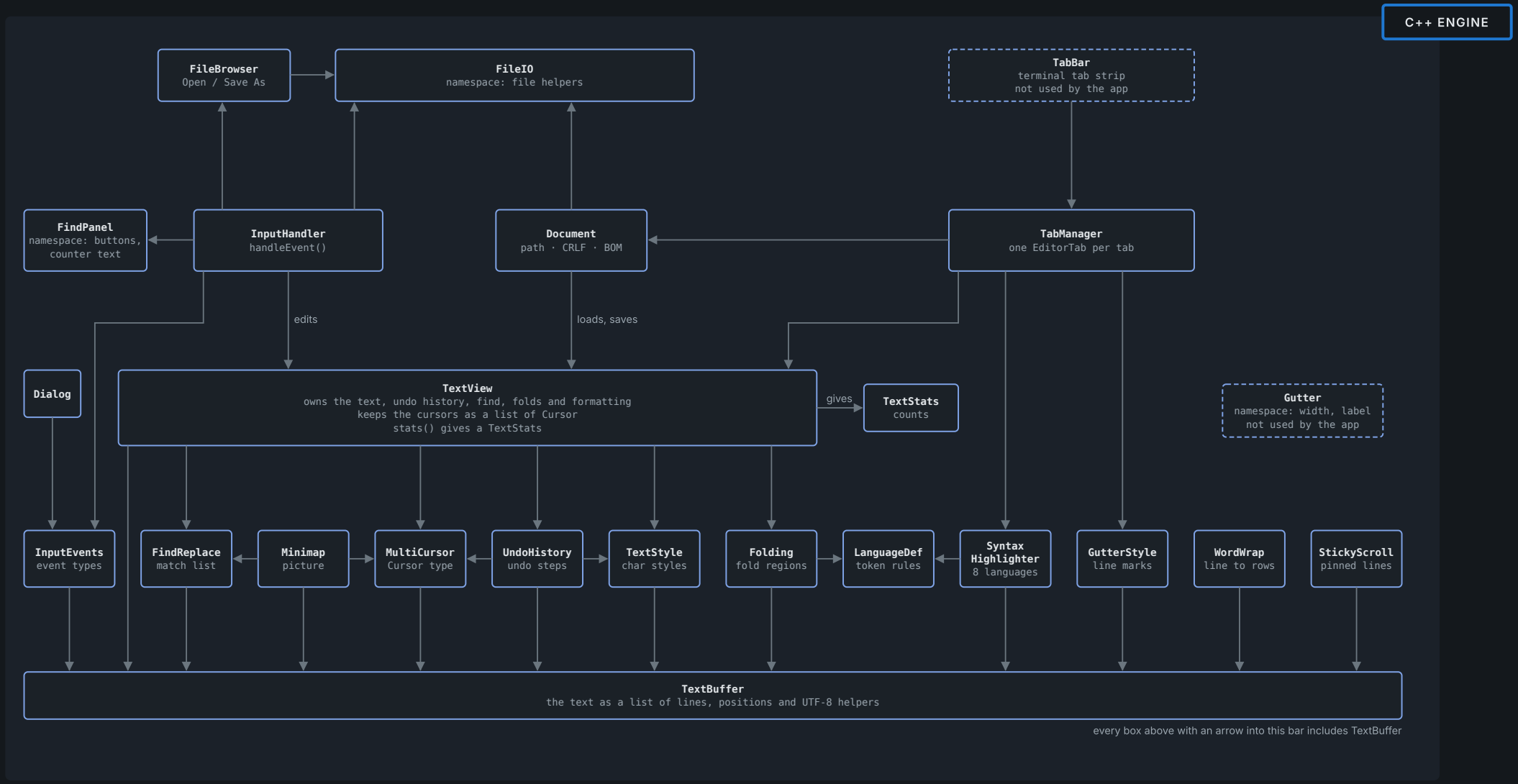
One keystroke, start to finish

1. `CodeEditor.qml` catches the key and calls `EditorController.keyPressed`.
2. `EditorController::feed` turns it into an `Input::Event` and passes it to `handleEvent(ev, view, state)` in the engine.
3. The engine edits the `TextView` and leaves requests in `InputState` (save, new tab, copy, quit).
4. `carryOut(requests)` does the Qt-side work for those requests: files, clipboard, tabs.
5. `rebuild(true)` asks `RowBuilder` to read the current tab and make rows of spans, selection, cursors, matches and gutter data.
6. `EditorRowsModel` updates and `rowsRebuilt` fires. `GutterController` refreshes `GutterModel` and `MinimapController` redraws on the same signal, and each `EditorRow` repaints.

Engine map: which module uses which

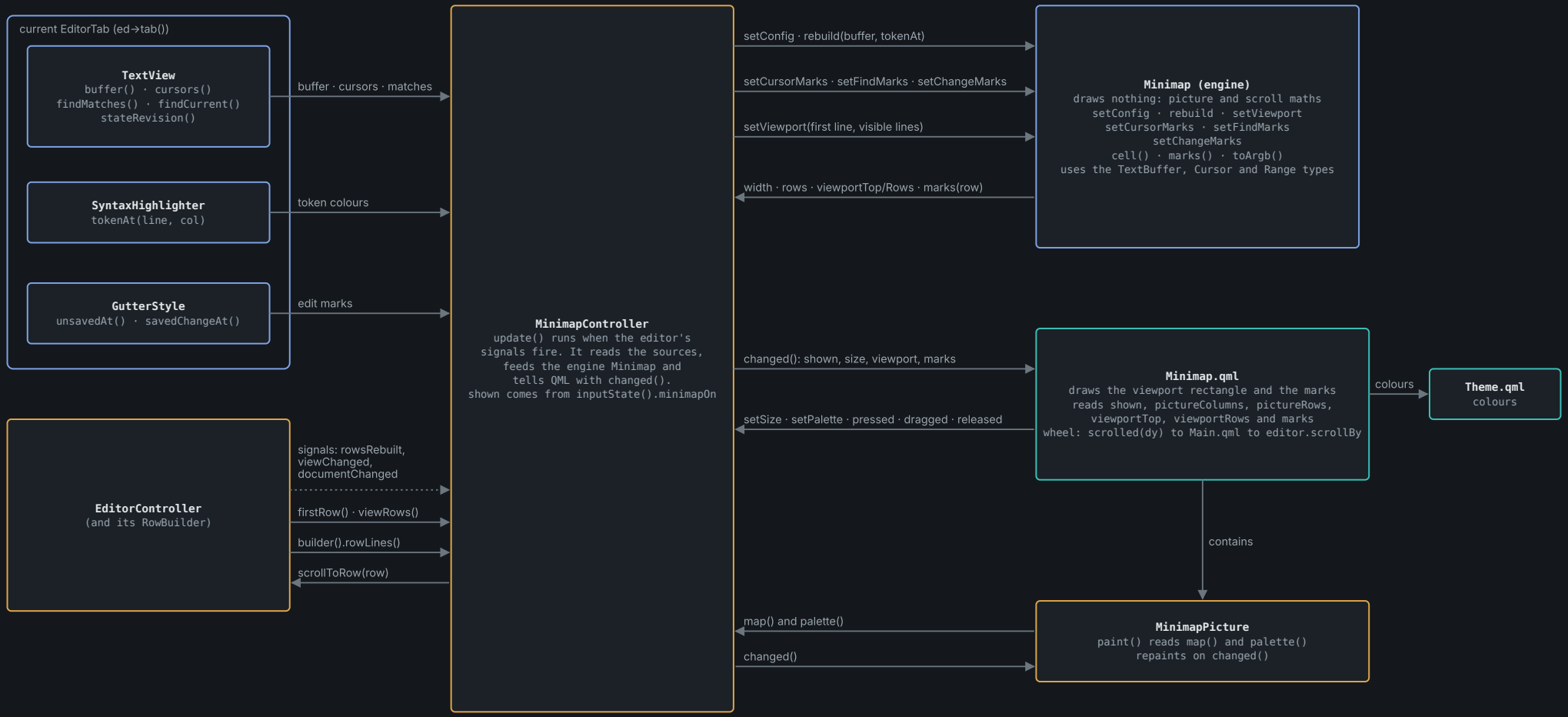
arrow: the module includes or uses the module it points to

dashed box: nothing includes it, so the app does not use it



Minimap flow: from the text to the pixels

arrow: the way the data or call goes · signal · blue: engine · orange: Qt wrapper · green: QML



Reading the engine map

An arrow means `#include` and use: the box at the tail needs the box at the head.

The base

- **`TextBuffer`** is the base: every box in the bottom row with an arrow into it includes it, and so does `TextView` (the long arrow on the left).
- **`LanguageDef`** is the only box in that row without an arrow into the bar, because it includes nothing.

The text and its helpers

- **`TextView`** holds a `Folding`, `TextBuffer`, `TextStyle`, `UndoHistory` and `FindReplace`, keeps its cursors as a list of `Cursor` (the type from `MultiCursor`), and builds a `TextStats` when asked.
- **`UndoHistory`** stores small changes with their text, formatting and cursors, which is why it points at `TextStyle` and `MultiCursor`.
- `Folding` and `SyntaxHighlighter` both read `LanguageDef`; `SyntaxHighlighter` also includes the eight language files (C/C++, JavaScript, Rust, QML, Fish, shell, CMake, Lua).

Tabs, files and input

- Each tab, held by **`TabManager`**, has a `TextView`, `Document`, `GutterStyle` and `SyntaxHighlighter`. `Document` loads and saves through `TextView` and `FileIO`.
- **`InputHandler`** edits the `TextView`, reads the events in `InputEvents`, reacts to clicks on the find bar through `FindPanel` and uses `FileBrowser` and `FileIO`. `Dialog` also uses `InputEvents`.
- **`Minimap`** is handed its data by the caller (a `TextBuffer`, a list of `Cursor`, the `Range` of each find match) and only uses those types.

Used by the Qt wrapper, and not drawn

- `WordWrap`, `StickyScroll` and `GutterStyle` are used by the Qt wrapper (`RowBuilder`, `EditorController`), not by other engine modules. `EditorController` gives `StickyScroll` its scope regions from `Folding::regions()`.
- `FindPanel`, `FileIO`, `WordWrap` and `Gutter` are **namespaces of functions**, not objects.
- The two dashed boxes, `Gutter` and `TabBar`, are not included by anything in the app.

Reading the minimap flow

`MinimapController` owns the engine `Minimap`. It wakes up when `EditorController` emits `rowsRebuilt`, `viewChanged` or `documentChanged` (the dotted arrow), then `update()` works in this order:

1. Reads `shown` from `inputState().minimapOn`, and stops if the minimap is off or has no size yet.
2. Calls **`rebuild(buffer, tokenAt)`**, giving the `Minimap` the text and a function that returns each character's syntax token from `SyntaxHighlighter`.
3. Sets the marks: the cursors and selections, the find matches and current match from `TextView`, and the unsaved and saved lines from `GutterStyle`.
4. Sets the visible part with **`setViewport`**, using `firstRow()` and `viewRows()` from `EditorController` and `rowLines()` from the `RowBuilder`.
5. Reads the size, viewport and marks back from the `Minimap` and emits **`changed()`**.

What the QML side does

- **`Minimap.qml`** shows the viewport rectangle and the marks from the controller's properties, and gets the syntax colours from `Theme.qml` through `setPalette`.
- **`MinimapPicture`**, inside `Minimap.qml`, reads the controller's `map()` and palette and paints the cells whenever `changed()` fires.
- A click or drag in the minimap calls `pressed`, `dragged` and `released` on the controller, which converts the minimap row to a line and calls **`EditorController::scrollToRow`**.
- The mouse wheel does not go through the controller: `Minimap.qml` emits `scrolled(dy)`, and `Main.qml` passes it to `editor.scrollBy`.